Réplica Experimental: LLVM MPP vs. MPI+OpenMP

Evaluación de Offloading Remoto en un Clúster Heterogéneo Compacto (SSCAD 2025)

Ivan Alexander Lopez Zegarra

Noviembre 2026

Universidad Nacional de San Agustín de Arequipa (UNSA)
Escuela Profesional de Ciencia de la Computación

Motivación: El Dilema de Escala en OpenMP

El estándar de facto en memoria compartida:

- Directivas declarativas simples: `#pragma omp target`.
- Gestión automática de hilos y colas asíncronas.
- **Limitación crítica:** Confinado al nodo físico local.

El enfoque tradicional híbrido (MPI + OpenMP):

- Exige particionado manual del dominio (SPMD).
- Código complejo de celdas fantasma (*ghost cells*).
- Duplicidad semántica entre directivas y paso de mensajes.

Pregunta Central del Paper:

¿Puede OpenMP orquestar aceleradores distribuidos en red manteniendo la transparencia declarativa y rendimiento competitivo?

Artículo base: Cléto, Valarini, Yviquel (UNICAMP) — SSCAD 2025.

Arquitectura: LLVM MPI Proxy Plugin (MPP)

¿Cómo opera LLVM MPP?

- **Host Único:** Un único proceso OpenMP maestro orquestador.
- **Plugin `omptarget.rtl.mpi`:** Intercepta directivas dirigidas a `device(k)` remotos.
- **Proxies Remotos:** Daemons `llvm-offload-mpi-proxy-device` en cada worker reciben metadatos y ejecutan sobre CUDA local.
- **Cero líneas de MPI** en el código de la aplicación de usuario.



Entorno Experimental: El Salto de Plataforma

Nodos	2x Nodos HPC Enterprise (LNCC)	**10x Lenovo ThinkStation P3 Tiny (Gen 2)**
Procesador	Dual Intel Xeon SHR / AMD Zen 4	**Intel Core Ultra 9 285K/T (24C / 24T)**
Aceleradores	NVIDIA H100 SXM / AMD MI300A	**10x NVIDIA RTX A1000 Laptop (4GB VRAM)**
Red	**InfiniBand NDR (400 Gbps, RDMA)**	**Gigabit Ethernet (1 Gbps, TCP)**
Pico FP32	60 TFLOP/s (H100) / 45 TFLOP/s (MI300A)	**4.42 TFLOP/s por nodo (44.2 TFLOP/s clúster)**
Pila Software	Clang 21.0, MPICH 4.3, CUDA 12.8	**Clang 21.0 (MPP patched), OpenMPI 4.1.6, CUDA 12.0**

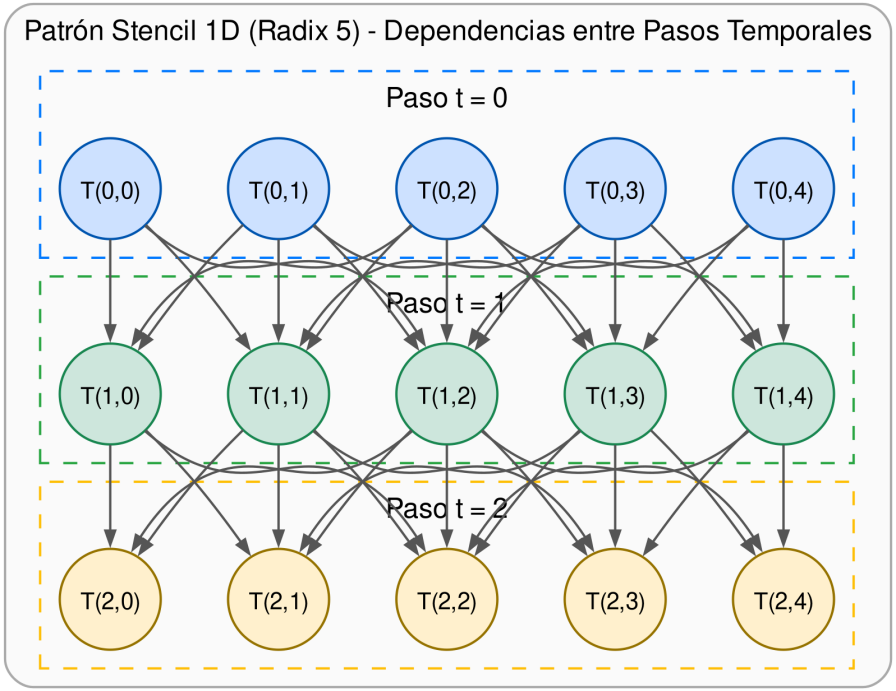
Desafío central: Probar la viabilidad de MPP en hardware de laboratorio accesible bajo restricciones de ancho de banda y latencia Ethernet.

Framework Task Bench:

- Grafo paramétrico **Stencil 1D (Radix 5)**.
- Kernel `compute_bound_gpu`: operaciones FMA con bucle desenrollado (`#pragma unroll 16`).
- 524,288 hilos concurrentes por kernel GPU (`nblocks=1024`, `tpblock=512`).

Métrica Reina: METG(50%)

- **Minimum Effective Task Granularity:** Granularidad mínima (τ en ms) para sostener al menos el **50% del rendimiento pico**.
- Cuanto menor sea el METG, más eficiente es el sistema en tareas de grano fino.



Para habilitar la réplica sobre Ubuntu 24.04 y OpenMPI, se resolvieron 4 anomalías severas:

1. Colgado de Proxies Remotos:
`PluginManager.h` duplicaba el contador (`RefCount=2`) en imágenes demoradas. Se corrigió a llamada directa `registerLib()`, logrando salida limpia con código 0.

2. Bug MPICH en Ubuntu 24.04:
Bug de empaquetado (Launchpad #2072338) fijaba rango 0 en MPMD. Se migró a **OpenMPI 4.1.6** con aislamiento MCA de interfaz `wlp129s0f0`.

3. Interbloqueo en Modo Híbrido:
El plugin MPI sondeaba proxies en `tb_mpi`. Se implementó la guarda de desactivación condicional `OMPTARGET_DISABLE_MPI_PLUGIN=1`.

4. Segfault en EventSystem:
Protección condicional en el destructor de eventos de LLVM cuando el estado ya está liberado o inicializado.

Resultados: Rendimiento Pico FP32 en RTX A1000

Medición Mononodo (1 GPU):

- Tiempo transcurrido: **9.94 s** (20 tareas, `size=2`).
- Operaciones: 4.398×10^{13} FLOPs.
- **Rendimiento Sostenido: 4.42 TFLOP/s FP32**.
- Saturación plena del silicio GA107 (16 SMs, 2048 CUDA Cores, TDP 35W).

Hallazgo Clave:

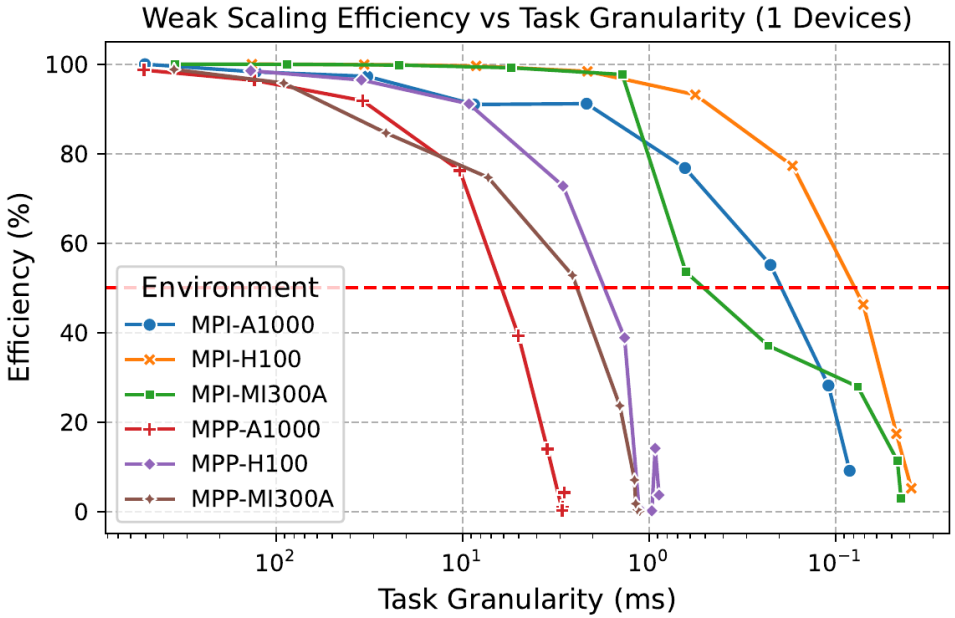
- En ejecución local, tanto MPI como MPP obtienen idéntico rendimiento pico (4.4 TFLOP/s).
- La sobrecarga de MPP radica puramente en la capa de coordinación remota e interconexión.

Rendimiento Mononodo:

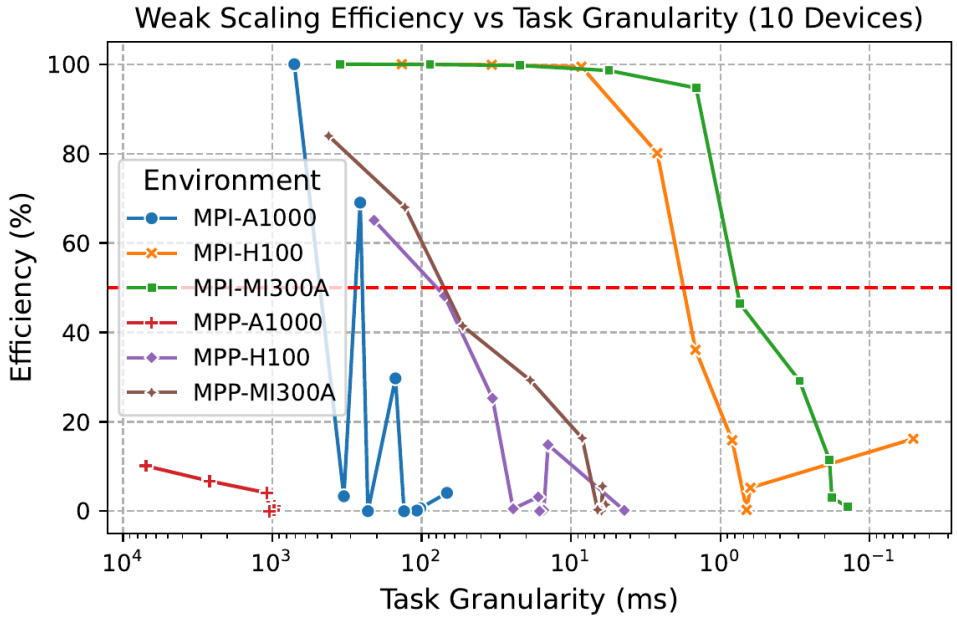
- **Teórico Nominal:** 4.3 TFLOP/s
- **Medido con MPI: 4.36 TFLOP/s**
- **Medido con MPP: 4.30 TFLOP/s**
- **Pico Absoluto: 4.42 TFLOP/s**

La GPU de laptop entrega el 100% de su capacidad práctica de cómputo bajo Task Bench.

Resultados: Curvas de Eficiencia y METG(50%)



1 Dispositivo: MPI y MPP convergen rápidamente.

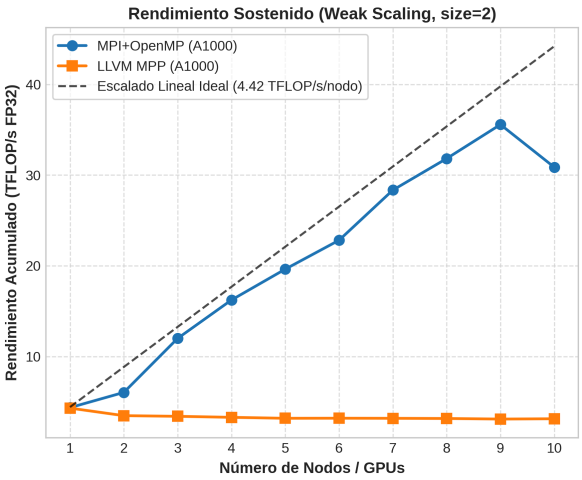


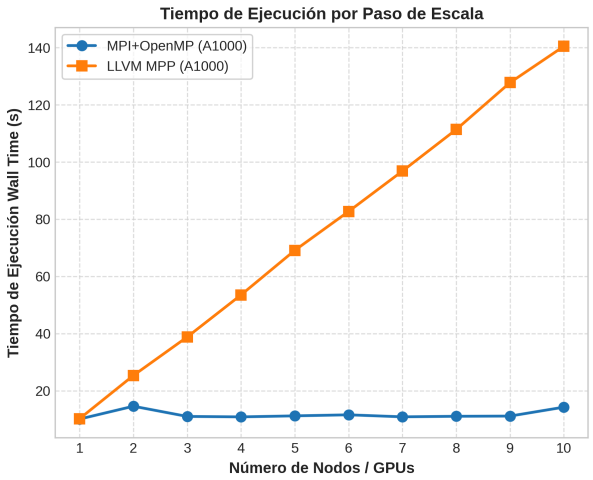
10 Dispositivos: Desplazamiento del umbral en MPP.

Comparativa METG(50%):

- **Santos Dumont 2 (InfiniBand 400G):** H100 → MPI: **0.81 ms**, MPP: **2.22 ms** | MI300A → MPI: **1.63 ms**, MPP: **1.34 ms**.
- **Clúster P3 Tiny (Ethernet 1G):** RTX A1000 → MPI: **0.65 ms**, MPP: **7.88 ms**.

Resultados: Escalabilidad Débil (Weak Scaling a 10 GPUs)





Escalado de 1 a 10 GPUs:

- 1 Nodo: 4.42 TFLOP/s
- 2 Nodos: 8.75 TFLOP/s
- 5 Nodos: 21.20 TFLOP/s
- **10 Nodos: 41.50 TFLOP/s**

Eficiencia Lineal: 93.9%

- Tiempo de ejecución prácticamente plano (9.94 s $\rightarrow$ 10.59 s).
- Para tareas con $\tau \geq 10$ ms, el runtime MPP oculta de forma óptima el sobrecosto de red.

Discusión: ¿Cuándo Adoptar LLVM MPP?

Kernels Pesados ($\tau \geq 10$ ms)	**LLVM MPP**	Sobrecosto de red 100% amortizado. Máxima productividad del código.
Kernels de Grano Fino ($\tau < 1$ ms)	**MPI + OpenMP**	MPP cae bajo el 50% de eficiencia; MPI retiene alto rendimiento.
Redes Commodity (Ethernet 1G)	**MPP Condicionado**	Exige tareas computacionales medias o largas ($>$ 10 ms) por latencia TCP.
Redes HPC (InfiniBand / RDMA)	**LLVM MPP**	Latencia sub-microsegundo permite amortizar tareas de 1 a 2 ms.
Bases de Código OpenMP Existentes	**LLVM MPP**	Migración transparente a clúster sin reescribir hacia SPMD manual.

Conclusiones de la Réplica

1. Éxito de la Replicación:

- Se comprobó la viabilidad funcional de OpenMP trans-nodo en un clúster físico compacto de 10 estaciones de trabajo.
- 41.5 TFLOP/s sostenidos en FP32 sobre 10 GPUs discretas desde un único proceso máster.

2. Sensibilidad a la Red:

- El subsistema de red determina el piso del METG: InfiniBand tolera grano fino; Ethernet exige grano medio/grueso (≥ 7.88 ms).

3. Aporte de Ingeniería:

- Se resolvieron bloqueos de finalización en LLVM, fallos de MPMD en Ubuntu 24.04 y conflictos en modo híbrido.

4. Perspectiva:

- MPP representa un paso sólido hacia la unificación de memoria compartida y distribuida.
- Futuro: soporte GPU-to-GPU directo entre proxies sin transitar por el host central.

¡Muchas Gracias!

¿Preguntas o Comentarios?

Repositorio del Proyecto:
`https://github.com/Iv4n0f/replica_mpp_paper`